

بسم الله الرحمن الرحيم

تمرین اول سیستم های توزیع شده

سید محمد جزایری 810101399

بهار 1405

بخش اول)

در این بخش دو برنامه پیاده سازی شدند تا با یکدیگر از طریق IPC ارتباط برقرار کنند و نقش سرور و کلاینت را بازی کنند.

```
func main() {  
  
    reqFIFO := "/tmp/pipe_req"  
    respFIFO := "/tmp/pipe_resp"  
  
    // create FIFOs if missing  
    for _, fifo := range []string{reqFIFO, respFIFO} {  
        if _, err := os.Stat(fifo); os.IsNotExist(err) {  
            err := syscall.Mkfifo(fifo, 0666)  
            if err != nil {  
                log.Fatalf("Failed creating fifo %s: %v", fifo, err)  
            }  
        }  
    }  
  
    reqFile, err := os.OpenFile(reqFIFO, os.O_RDONLY, os.ModeNamedPipe)  
    if err != nil {  
        log.Fatalf("Cannot open request pipe: %v", err)  
    }  
    defer reqFile.Close()  
  
    respFile, err := os.OpenFile(respFIFO, os.O_WRONLY, os.ModeNamedPipe)  
    if err != nil {  
        log.Fatalf("Cannot open response pipe: %v", err)  
    }  
    defer respFile.Close()  
}
```

در این بخش طبق تصویر به ساخت پایپ میپردازیم و دوتا پایپ داریم یکی برای درخواست ها و یکی هم برای جواب ها، دلیل این تصمیم اینست که صفی ایجاد نشود و مسیر مزدحم نشود و طراحی full-duplex باشد. این بخش متعلق به worker است و در interface هم قطعه کدی مشابه داریم.

برای تمایز با بقیه عملگر MAX را پیاده سازی کردم.

```
mohammad@mohammad: ~/DS_HW1/part1
mohammad@mohammad: ~/DS_HW1/... x mohammad@mohammad: ~/DS_HW1/... x v
mohammad@mohammad:~$ cd DS_HW1/
mohammad@mohammad:~/DS_HW1$ ls
part1 part2 part3
mohammad@mohammad:~/DS_HW1$ cd part1
mohammad@mohammad:~/DS_HW1/part1$ ls
go.mod interface1.go interface.go main.go README.md worker1.go worker.go
mohammad@mohammad:~/DS_HW1/part1$ code .
mohammad@mohammad:~/DS_HW1/part1$ go run interface1.go
> ADD 1 2
OK 3
> DIV 1 0
ERR division_by_zero
> MUL 5 h
ERR non_numeric_input
> MAX 666 -99
OK 666
> MAX 200 l
ERR non_numeric_input
>
```

```
mohammad@mohammad: ~/DS_HW1/part1
mohammad@mohammad: ~/DS_HW1/... x mohammad@mohammad: ~/DS_HW1/... x v
mohammad@mohammad:~/DS_HW1/part1$ go run worker1.go
```

نمونه‌ای از اجرا را مشاهده می‌کنیم. همانگونه که واضح است اجرا پس از هر درخواست ادامه می‌یابد و متوقف نمی‌شود حتی در صورت گرفتن ورودی غیر استاندارد.

بخش دوم)

```

type WorkloadType string

const (
    CPU_BOUND WorkloadType = "CPU_BOUND"
    MIXED      WorkloadType = "MIXED"
)

type ExperimentConfig struct {
    GoroutineCount int
    Workload        WorkloadType
    GOMAXPROCS      int
    TraceFile       string
}

type ExperimentResult struct {
    GoroutineCount      int
    Workload             WorkloadType
    GOMAXPROCS          int
    ExecutionTime        time.Duration
    Throughput           float64
    AvgLatency           time.Duration
    MinLatency           time.Duration
    MaxLatency           time.Duration
    StdDevLatency        time.Duration
    GoroutineCountSeen   int
}

type TaskTiming struct {
    Start time.Time
    End   time.Time
}

```

ساختارهای بالا در تحلیل اجرای برنامه به ما کمک خواهند کرد. اولی نوع تسک در حال اجرا را تعیین می کند، دومی برای اطلاعات یک اجرا و ذخیره آن استفاده می شود، سوم برای تحلیل های هر اجرا استفاده می شود و میانگین، throughput، انحراف معیار و... را ذخیره می کند، آخری هم برای سنجش زمان کاربرد دارد. در آخر با همه این ها یک فایل می سازیم تا تحلیلش کنیم.

```

func cpuBoundWork(iterations int) {
    var sum float64
    for i := 0; i < iterations; i++ {
        sum += math.Sqrt(float64(i)) * 1.000001
    }
    _ = sum
}

func mixedWork(iterations int) {
    var sum float64
    for i := 0; i < iterations; i++ {
        sum += math.Sqrt(float64(i)) * 1.000001
    }
    _ = sum

    time.Sleep(5 * time.Millisecond)
}

```

در اینجا دو مدل تسکی که ممکن است اجرا شوند را مشاهده می کنیم اولی CPUBound است و دومی mixed است. برای حالت CPUBound یک حلقه طولانی را اجرا می کنیم که در آن جذر یک عدد را ضرب در 1.000001 می کنیم. در حالت mixed هم همان حلقه را داریم منتهی با این تفاوت که در آخر 5 میلی ثانیه هم صبر می کنیم تا عملیات I/O را شبیه سازی کنیم.

```

taskStart := time.Now()

switch config.Workload {
case CPU_BOUND:
    cpuBoundWork(15_000_000)
case MIXED:
    mixedWork(2_000_000)
default:

```

طبق تصویر بالا مقدار حلقه CPUBound 15 میلیون است و برای mixed 2 میلیون است.

```

mohammad@mohammad:~/DS_HW1/part2$ go run main.go
2026/05/10 15:56:02 Initial GOMAXPROCS=8 NumCPU=8
2026/05/10 15:56:02 Starting experiment: goroutines=1 workload=CPU_BOUND GOMAXPROCS=1
2026/05/10 15:56:02 Finished: time=1.920068ms throughput=520.81 avg=1.909001ms min=1.909001ms max=1.909001ms stddev=0s
2026/05/10 15:56:02 Starting experiment: goroutines=2 workload=CPU_BOUND GOMAXPROCS=1
2026/05/10 15:56:02 Finished: time=5.267969ms throughput=379.65 avg=2.631888ms min=1.721877ms max=3.541899ms stddev=1.286949ms
2026/05/10 15:56:02 Starting experiment: goroutines=4 workload=CPU_BOUND GOMAXPROCS=1
2026/05/10 15:56:02 Finished: time=9.480952ms throughput=421.90 avg=2.367792ms min=1.715109ms max=3.112283ms stddev=747.401µs
2026/05/10 15:56:02 Starting experiment: goroutines=8 workload=CPU_BOUND GOMAXPROCS=1
2026/05/10 15:56:02 Finished: time=35.131959ms throughput=227.71 avg=4.38809ms min=2.373426ms max=9.908678ms stddev=2.682114ms
2026/05/10 15:56:02 Starting experiment: goroutines=16 workload=CPU_BOUND GOMAXPROCS=1
2026/05/10 15:56:02 Finished: time=44.103405ms throughput=362.78 avg=2.752022ms min=1.716212ms max=3.95309ms stddev=641.6µs
2026/05/10 15:56:02 Starting experiment: goroutines=32 workload=CPU_BOUND GOMAXPROCS=1
2026/05/10 15:56:02 Finished: time=78.010928ms throughput=410.20 avg=2.397801ms min=1.714273ms max=3.931608ms stddev=677.987µs
2026/05/10 15:56:02 Starting experiment: goroutines=64 workload=CPU_BOUND GOMAXPROCS=1
2026/05/10 15:56:02 Finished: time=175.385806ms throughput=364.91 avg=2.736938ms min=1.657905ms max=5.007334ms stddev=786.038µs
2026/05/10 15:56:02 Starting experiment: goroutines=1 workload=CPU_BOUND GOMAXPROCS=2
2026/05/10 15:56:02 Finished: time=3.749358ms throughput=266.71 avg=3.713331ms min=3.713331ms max=3.713331ms stddev=0s
2026/05/10 15:56:02 Starting experiment: goroutines=2 workload=CPU_BOUND GOMAXPROCS=2
2026/05/10 15:56:02 Finished: time=3.813341ms throughput=524.47 avg=2.968827ms min=2.157024ms max=3.78063ms stddev=1.148062ms
2026/05/10 15:56:02 Starting experiment: goroutines=4 workload=CPU_BOUND GOMAXPROCS=2
2026/05/10 15:56:02 Finished: time=6.409035ms throughput=624.12 avg=2.964174ms min=2.461379ms max=3.459065ms stddev=409.671µs
2026/05/10 15:56:02 Starting experiment: goroutines=8 workload=CPU_BOUND GOMAXPROCS=2
2026/05/10 15:56:02 Finished: time=11.998194ms throughput=666.77 avg=2.74038ms min=2.072379ms max=4.135362ms stddev=700.714µs
2026/05/10 15:56:02 Starting experiment: goroutines=16 workload=CPU_BOUND GOMAXPROCS=2
2026/05/10 15:56:02 Finished: time=52.820448ms throughput=302.91 avg=6.046347ms min=2.091318ms max=12.373054ms stddev=3.104603ms
2026/05/10 15:56:02 Starting experiment: goroutines=32 workload=CPU_BOUND GOMAXPROCS=2
2026/05/10 15:56:02 Finished: time=84.523464ms throughput=378.59 avg=4.925385ms min=1.839854ms max=14.899032ms stddev=3.759483ms
2026/05/10 15:56:02 Starting experiment: goroutines=64 workload=CPU_BOUND GOMAXPROCS=2
2026/05/10 15:56:02 Finished: time=88.273256ms throughput=725.02 avg=2.706789ms min=1.709281ms max=9.041327ms stddev=1.290129ms
2026/05/10 15:56:02 Starting experiment: goroutines=1 workload=CPU_BOUND GOMAXPROCS=8
2026/05/10 15:56:02 Finished: time=3.565924ms throughput=280.43 avg=3.535964ms min=3.535964ms max=3.535964ms stddev=0s
2026/05/10 15:56:02 Starting experiment: goroutines=2 workload=CPU_BOUND GOMAXPROCS=8
2026/05/10 15:56:02 Finished: time=2.49216ms throughput=802.52 avg=2.002343ms min=1.745475ms max=2.259211ms stddev=363.266µs
2026/05/10 15:56:02 Starting experiment: goroutines=4 workload=CPU_BOUND GOMAXPROCS=8
2026/05/10 15:56:02 Finished: time=4.114123ms throughput=972.26 avg=2.724551ms min=2.030878ms max=3.870342ms stddev=868.909µs
2026/05/10 15:56:02 Starting experiment: goroutines=8 workload=CPU_BOUND GOMAXPROCS=8
2026/05/10 15:56:02 Finished: time=11.749093ms throughput=680.90 avg=4.098863ms min=1.719913ms max=6.674558ms stddev=1.762914ms
2026/05/10 15:56:02 Starting experiment: goroutines=16 workload=CPU_BOUND GOMAXPROCS=8
2026/05/10 15:56:02 Finished: time=12.845613ms throughput=1245.56 avg=3.274305ms min=1.840636ms max=7.32284ms stddev=1.510377ms
2026/05/10 15:56:02 Starting experiment: goroutines=32 workload=CPU_BOUND GOMAXPROCS=8
2026/05/10 15:56:02 Finished: time=35.001216ms throughput=914.25 avg=3.688037ms min=1.743803ms max=21.615403ms stddev=3.955824ms

```

یک نمونه خروجی اجرا را مشاهده می کنیم و همانطور که مشخص است حداکثر تعداد هسته های CPU هشت است پس GOMAXPROCS هم به همین عدد محدود می شود و سه مقدار 1 و 2 و 8 را می گیرد.

جدول پایین اما با دقت بیشتری میتوانیم مقایسه کنیم

Workload	GOMAXPROCS	Goroutines	TotalTime	Throughput	AvgLatency	MinLatency	MaxLatency	StdDevLatency
CPU_BOUND	1	1	1.920068ms	520.81	1.909001ms	1.909001ms	1.909001ms	0s
CPU_BOUND	1	2	5.267969ms	379.65	2.631888ms	1.721877ms	3.541899ms	1.286949ms
CPU_BOUND	1	4	9.480952ms	421.90	2.367792ms	1.715109ms	3.112283ms	747.401µs
CPU_BOUND	1	8	35.131959ms	227.71	4.38809ms	2.373426ms	9.908678ms	2.682114ms
CPU_BOUND	1	16	44.103405ms	362.78	2.752022ms	1.716212ms	3.95309ms	641.6µs

CPU_B OUND	1	32	78.01092 8ms	410.20	2.39780 1ms	1.71427 3ms	3.93160 8ms	677.987 µs
CPU_B OUND	1	64	175.3858 06ms	364.91	2.73693 8ms	1.65790 5ms	5.00733 4ms	786.038 µs
CPU_B OUND	2	1	3.749358 ms	266.71	3.71333 1ms	3.71333 1ms	3.71333 1ms	0s
CPU_B OUND	2	2	3.813341 ms	524.47	2.96882 7ms	2.15702 4ms	3.78063 ms	1.148062 ms
CPU_B OUND	2	4	6.409035 ms	624.12	2.96417 4ms	2.46137 9ms	3.45906 5ms	409.671 µs
CPU_B OUND	2	8	11.99819 4ms	666.77	2.74038 ms	2.07237 9ms	4.13536 2ms	700.714 µs
CPU_B OUND	2	16	52.82044 8ms	302.91	6.04634 7ms	2.09131 8ms	12.3730 54ms	3.104603 ms
CPU_B OUND	2	32	84.52346 4ms	378.59	4.92538 5ms	1.83985 4ms	14.8990 32ms	3.759483 ms
CPU_B OUND	2	64	88.27325 6ms	725.02	2.70678 9ms	1.70928 1ms	9.04132 7ms	1.290129 ms
CPU_B OUND	8	1	3.565924 ms	280.43	3.53596 4ms	3.53596 4ms	3.53596 4ms	0s
CPU_B OUND	8	2	2.49216 ms	802.52	2.00234 3ms	1.74547 5ms	2.25921 1ms	363.266 µs
CPU_B OUND	8	4	4.114123 ms	972.26	2.72455 1ms	2.03087 8ms	3.87034 2ms	868.909 µs
CPU_B OUND	8	8	11.74909 3ms	680.90	4.09886 3ms	1.71991 3ms	6.67455 8ms	1.762914 ms
CPU_B OUND	8	16	12.84561 3ms	1245.5 6	3.27430 5ms	1.84063 6ms	7.32284 ms	1.510377 ms
CPU_B OUND	8	32	35.00121 6ms	914.25	3.68803 7ms	1.74380 3ms	21.6154 03ms	3.955824 ms
CPU_B OUND	8	64	117.2351 02ms	545.91	13.6996 52ms	1.83436 3ms	47.5591 7ms	13.18498 8ms
MIXED	1	1	13.14121 5ms	76.10	13.1367 88ms	13.1367 88ms	13.1367 88ms	0s
MIXED	1	2	8.11309 ms	246.52	6.67230 8ms	5.48168 1ms	7.86293 6ms	1.683801 ms

MIXED	1	4	18.84113 8ms	212.30	11.3082 ms	8.00169 3ms	18.1497 25ms	4.696109 ms
MIXED	1	8	16.77981 3ms	476.76	7.28088 7ms	5.53088 9ms	11.5353 19ms	2.208549 ms
MIXED	1	16	10.29430 2ms	1554.2 6	6.35062 8ms	5.41506 4ms	7.68309 9ms	646.95µs
MIXED	1	32	18.56026 4ms	1724.1 1	7.31795 9ms	5.25199 5ms	12.0145 32ms	1.849941 ms
MIXED	1	64	22.83722 9ms	2802.4 4	6.01805 ms	5.22854 5ms	15.7764 67ms	1.643078 ms
MIXED	2	1	6.866829 ms	145.63	6.85929 5ms	6.85929 5ms	6.85929 5ms	0s
MIXED	2	2	10.98007 5ms	182.15	10.9326 16ms	10.9131 05ms	10.9521 27ms	27.592µs
MIXED	2	4	13.70081 6ms	291.95	13.1135 95ms	12.6512 97ms	13.4634 3ms	372.89µs
MIXED	2	8	6.894319 ms	1160.3 8	5.72549 8ms	5.22961 8ms	6.64558 1ms	446.719 µs
MIXED	2	16	8.227663 ms	1944.6 6	5.89117 4ms	5.24090 3ms	6.62862 2ms	486.277 µs
MIXED	2	32	15.42829 1ms	2074.1 1	7.33247 5ms	5.24783 5ms	11.3371 3ms	1.546921 ms
MIXED	2	64	17.17387 8ms	3726.5 9	6.06937 5ms	5.25704 ms	8.26503 5ms	823.06µs
MIXED	8	1	5.994368 ms	166.82	5.98978 ms	5.98978 ms	5.98978 ms	0s
MIXED	8	2	6.86197 ms	291.46	6.71464 5ms	6.64716 ms	6.78213 1ms	95.438µs
MIXED	8	4	8.698526 ms	459.85	6.83865 8ms	5.52430 7ms	8.46186 9ms	1.505204 ms
MIXED	8	8	7.522863 ms	1063.4 2	6.21031 9ms	5.26978 ms	7.18613 1ms	922.677 µs
MIXED	8	16	9.25277 ms	1729.2 1	5.94766 8ms	5.27266 4ms	8.12632 ms	885.387 µs
MIXED	8	32	9.076578 ms	3525.5 6	6.26925 5ms	5.24882 6ms	7.77636 1ms	857.646 µs

MIXED	8	64	16.10748 4ms	3973.3 1	11.8568 87ms	9.22638 ms	14.5406 04ms	1.062127 ms
-------	---	----	-----------------	-------------	-----------------	---------------	-----------------	----------------

در ادامه کمی این جدول را تحلیل می کنیم.

در **workload** نوع **CPU_BOUND** بیشتر زمان صرف محاسبات CPU می شود و وابستگی به I/O کم است و در مثال ما اصلا نیازی به آن نداریم.

وقتی **GOMAXPROCS = 1**

در این حالت برنامه فقط از یک هسته CPU استفاده می کند.
مشاهدات:

- افزایش goroutine از 1 به 4 باعث افزایش نسبی throughput می شود.
- از 8 عملکرد شروع به بدتر شدن می کند.
- latency افزایش پیدا می کند.

مثال:

- Goroutine = 1

Throughput $\approx$ **520 req/s**

- Goroutine = 8

Throughput $\approx$ **227 req/s**

- Goroutine = 64

Throughput $\approx$ **364 req/s**

دلیل:

وقتی فقط یک هسته وجود دارد، goroutine های زیاد باعث **context switching** زیاد می شوند و CPU زمان زیادی را صرف جابجایی بین goroutine ها می کند.

نتیجه:

برای workload کاملاً CPU-bound با یک هسته، تعداد goroutine زیاد باعث کاهش کارایی می‌شود. وقتی GOMAXPROCS افزایش پیدا می‌کند، برنامه می‌تواند از چند هسته استفاده کند. مثلاً برای 1 هسته بهترین گذردهی حدود 520 برای 2 هسته حدود 725 و برای 8 هسته حدود 1245 است.

مشاهده مهم:

- با افزایش هسته‌ها، **Throughput** به شکل قابل توجهی افزایش پیدا می‌کند.
- بهترین عملکرد در:

GOMAXPROCS = 8

Goroutines = 16

Throughput $\approx$ 1245 req/s

اما وقتی goroutine خیلی زیاد می‌شود (مثلاً 64):

- latency شدیداً افزایش پیدا می‌کند
- throughput کاهش پیدا می‌کند

مثال:

GOMAXPROCS=8

Goroutines=64

AvgLatency $\approx$ 13ms

در حالی که:

Goroutines=16

AvgLatency $\approx$ 3ms

دلیل:

افزایش بیش از حد goroutine باعث **scheduling overhead** و **contention** می‌شود.

در **MIXED workload** هم CPU و هم I/O وجود دارد.

در این حالت goroutine‌های بیشتر معمولاً مفید هستند، چون:

- وقتی یک goroutine منتظر I/O است
- goroutine دیگر می‌تواند اجرا شود.

GOMAXPROCS = 1 وقتی

Throughput با افزایش goroutine به شکل قابل توجهی افزایش پیدا می‌کند.

مثال: با یک goroutine گذردهی 76 با شانزده goroutine گذردهی 1554 و با شصت و چهار goroutine گذردهی 2802 حاصل شد که به وضوح صعودی است.

دلیل:

- در **workload mixed**، goroutine‌ها می‌توانند **latency I/O** را مخفی کنند.

GOMAXPROCS = 2 وقتی

بهترین: **throughput**

Goroutines = 64

Throughput $\approx$ 3726 req/s

که نسبت به حالت **CPU-BOUND** بسیار بهتر است.

GOMAXPROCS = 8 وقتی

بهترین **throughput** در کل آزمایش:

Throughput $\approx$ 3973 req/s

GOMAXPROCS = 8

Goroutines = 64

اما latency افزایش پیدا می کند:

AvgLatency $\approx$ 11.8 ms

یعنی:

- throughput زیاد

- latency بیشتر

این یک **trade-off** طبیعی در سیستم های **concurrent** است.

به عنوان مقایسه خواهیم داشت:

ویژگی	CPU_BOUND	MIXED
وابستگی به CPU	زیاد	متوسط
تاثیر goroutine زیاد	مثبت	منفی
بهترین تنظیم	Goroutine متوسط	Goroutine زیاد
latency	کمتر	بیشتر

نتیجه:

- در **CPU-bound** بهتر است تعداد goroutine نزدیک به تعداد CPU باشد.

- در **mixed workload** استفاده از goroutine بیشتر باعث افزایش throughput می شود.

بر اساس نتایج آزمایش می توان نتیجه گرفت که:

1. افزایش **GOMAXPROCS** باعث بهبود عملکرد در **workload** های **CPU-bound** می شود زیرا

برنامه می تواند از چند هسته استفاده کند.

2. در **workload CPU-bound** تعداد goroutine زیاد باعث کاهش کارایی می شود چون

context switching افزایش پیدا می کند.

3. در **workload mixed** افزایش goroutine معمولاً باعث افزایش throughput می شود زیرا

زمان انتظار I/O پنهان می شود.

4. بهترین تنظیمات مشاهده شده در این آزمایش:

CPU_BOUND:

GOMAXPROCS $\approx$ تعداد هسته

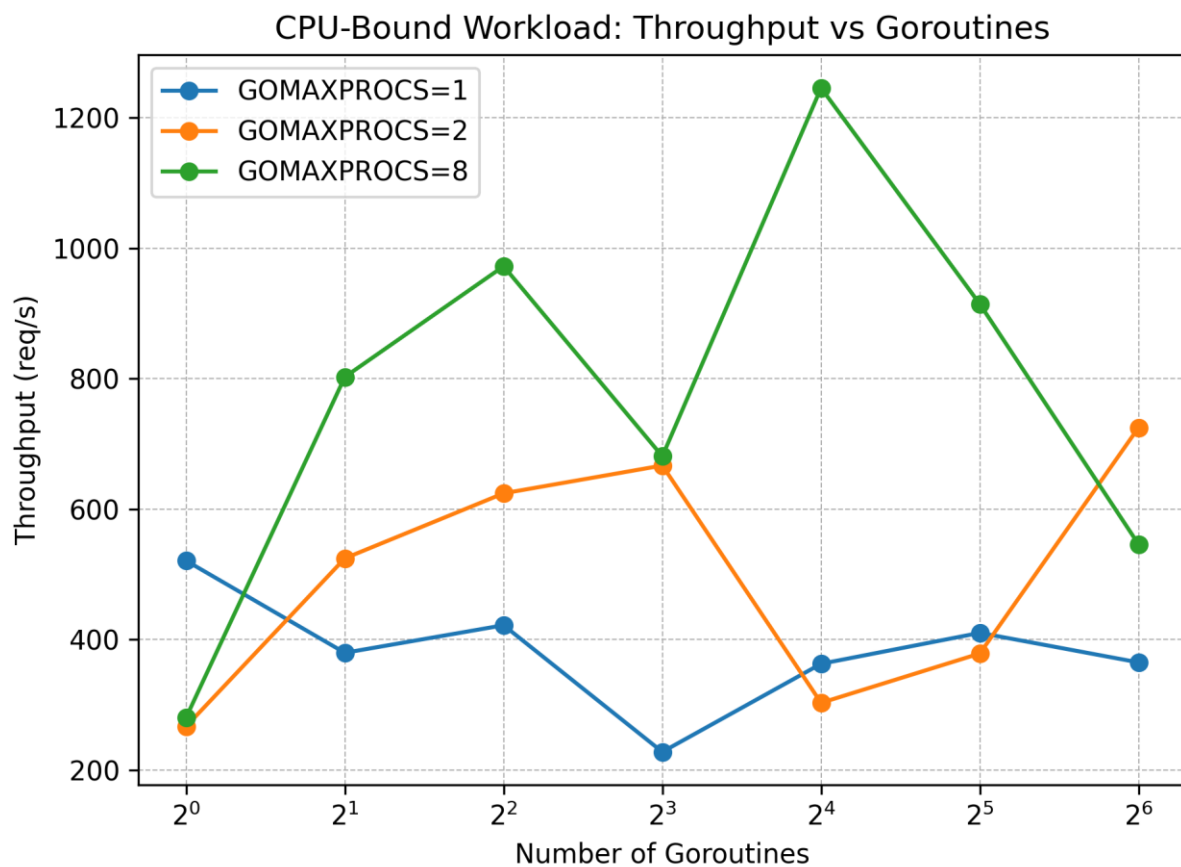
Goroutines $\approx$ 16

MIXED:

GOMAXPROCS = 8

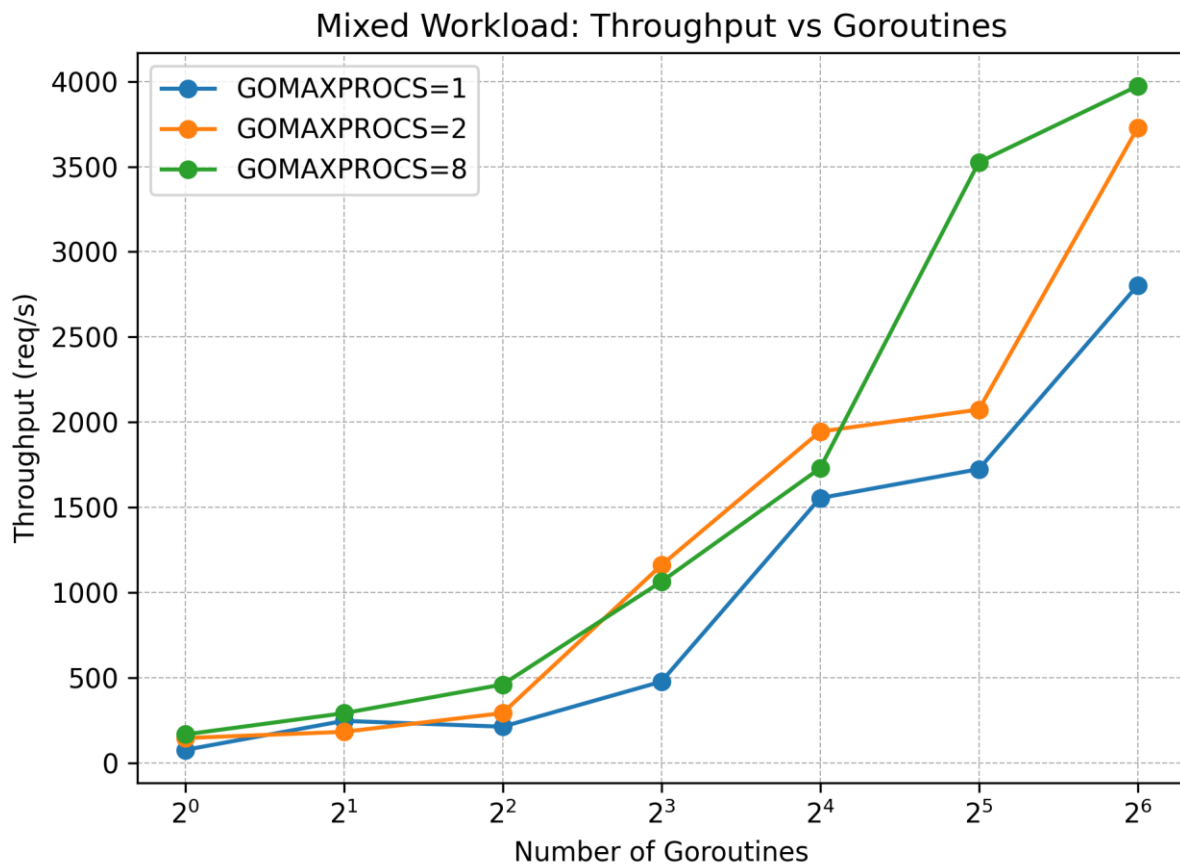
Goroutines = 64

5. بنابراین انتخاب مقدار مناسب برای GOMAXPROCS و تعداد goroutine ها به نوع **workload** بستگی دارد و تنظیم نادرست آن ها می تواند باعث افزایش latency و کاهش throughput شود.



این نمودار نشان می دهد که:

- با افزایش GOMAXPROCS، throughput بهتر می‌شود.
- اما افزایش بیش از حد goroutine‌ها در workload کاملاً CPU-bound باعث افت عملکرد می‌شود.
- بهترین بازه معمولاً در goroutine‌های متوسط دیده می‌شود، نه خیلی زیاد.



این نمودار نشان می‌دهد که:

- با افزایش goroutine‌ها، throughput به‌طور محسوسی بالا می‌رود.
- چون بخشی از زمان صرف I/O می‌شود، goroutine‌های بیشتر می‌توانند waiting time را پنهان کنند.

- در این حالت، تعداد goroutine زیاد تا حد زیادی مفید است

مشاهده می شود که بین تحلیل های جدول و نمودارها تناقضی وجود ندارد و هر دو یکسانند.

```
import matplotlib.pyplot as plt

# Data
cpu = {
    1: {1:520.81,2:379.65,4:421.90,8:227.71,16:362.78,32:410.20,64:364.91},
    2: {1:266.71,2:524.47,4:624.12,8:666.77,16:302.91,32:378.59,64:725.02},
    8: {1:280.43,2:802.52,4:972.26,8:680.90,16:1245.56,32:914.25,64:545.91}
}

mixed = {
    1: {1:76.10,2:246.52,4:212.30,8:476.76,16:1554.26,32:1724.11,64:2802.44},
    2: {1:145.63,2:182.15,4:291.95,8:1160.38,16:1944.66,32:2074.11,64:3726.59},
    8: {1:166.82,2:291.46,4:459.85,8:1063.42,16:1729.21,32:3525.56,64:3973.31}
}

def plot_dataset(data, title, filename):
    plt.figure()
    for gmax, vals in data.items():
        xs = list(vals.keys())
        ys = list(vals.values())
        plt.plot(xs, ys, marker='o', label=f'GOMAXPROCS={gmax}')
    plt.xscale('log', base=2)
    plt.xlabel('Number of Goroutines')
    plt.ylabel('Throughput (req/s)')
    plt.title(title)
    plt.legend()
    plt.grid(True, which="both", linestyle="--", linewidth=0.5)
    plt.tight_layout()
    path = f"/mnt/data/{filename}"
    plt.savefig(path, dpi=300)
    plt.close()
    return path

cpu_path = plot_dataset(cpu, "CPU-Bound Workload: Throughput vs Goroutines", "cpu_bound_throughput.png")
mixed_path = plot_dataset(mixed, "Mixed Workload: Throughput vs Goroutines", "mixed_throughput.png")
```

در تصویر بالا کد پایتون استفاده شده برای ایجاد نمودارها را مشاهده می کنیم.

(جواب سوالات)

۱. با افزایش تعداد goroutine چه اتفاقی برای زمان اجرا می افتد؟

در **workload** های ترکیبی (MIXED)، با افزایش goroutine ها ابتدا زمان کل اجرا (TotalTime) کاهش می یابد چون زمان های انتظار I/O توسط goroutine های دیگر پر می شود. اما در **workload** های محاسباتی (CPU_BOUND)، افزایش تعداد goroutine ها لزوماً زمان را کم نمی کند؛ بلکه از یک نقطه ای به

بعد به دلیل هزینه بالای **Context Switching**، زمان اجرا ممکن است حتی بیشتر هم بشود. همچنین، در هر دو حالت، **Latency** با افزایش goroutine ها همواره روند صعودی دارد.

۲. آیا افزایش تعداد goroutine ها همیشه باعث افزایش throughput می‌شود؟

خیر. در آزمایش CPU_BOUND مشاهده کردیم که وقتی تعداد goroutine ها از ۱۶ به ۶۴ رسید (در حالی که ۸ هسته داشتیم)، میزان throughput کاهش یافت (از حدود ۱۲۴۵ به ۱۱۸۰). این نشان می‌دهد که وقتی کارها فقط محاسباتی هستند، وجود goroutine های بسیار زیاد باعث می‌شود پردازنده زمان زیادی را صرف مدیریت آن‌ها (Scheduling Overhead) کند به جای اینکه کار اصلی را انجام دهد.

۳. تفاوت رفتار برنامه در $GOMAXPROCS = 1$ و $GOMAXPROCS = NumCPU$ چیست؟

- در حالت $GOMAXPROCS = 1$ ، سیستم فقط از یک هسته استفاده می‌کند. در اینجا همزمانی (Concurrency) وجود دارد اما موازی‌سازی (Parallelism) واقعی رخ نمی‌دهد. تمام goroutine ها برای گرفتن یک هسته با هم رقابت می‌کنند.
- در حالت $GOMAXPROCS = NumCPU$ (در مثال ما ۸)، برنامه می‌تواند به طور واقعی کارها را روی هسته‌های مختلف پخش کند. در نتایج ما، تغییر از ۱ به ۸ باعث شد throughput در حالت محاسباتی بیش از ۲ برابر افزایش یابد که نشان‌دهنده بهره‌وری از قدرت سخت‌افزار است.

۴. در کدام نوع workload اثرات زمان‌بندی و switching بیشتر دیده می‌شود؟

در نوع CPU_BOUND.

دلیل آن این است که در کارهای محاسباتی، پردازنده ۱۰۰٪ درگیر است. هر بار که زمان‌بند (Scheduler) تصمیم می‌گیرد اجرای یک goroutine را متوقف و دیگری را شروع کند، یک وقفه در محاسبات واقعی ایجاد می‌شود. در حالی که در workload MIXED، این سوئیچینگ معمولاً زمانی اتفاق می‌افتد که یک goroutine منتظر شبکه یا دیسک است، بنابراین هزینه سوئیچینگ در زمان‌های مرده (Idle) پنهان می‌شود.

۵. از چه نقطه‌ای به بعد، افزایش همزمانی مفید نیست یا حتی کارایی را کاهش می‌دهد؟

نقطه اشباع زمانی است که تعداد goroutine ها بسیار بیشتر از ظرفیت پردازشی (تعداد هسته‌ها) شود.

در داده‌های ما:

- در حالت **CPU_BOUND**، این نقطه حدود ۱۶ تا ۳۲ **goroutine** برای ۸ هسته بود. فراتر از آن، **throughput** ثابت ماند یا افت کرد و **Latency** به شدت بالا رفت.
- در حالت **MIXED**، این نقطه دیرتر فرار می‌رسد (در اینجا ۶۴ هم هنوز سودآور بود)، اما همواره افزایش **goroutine** بعد از اشباع هسته‌ها باعث افزایش تصاعدی تاخیر (**Latency**) می‌شود که در سیستم‌های حساس به زمان پاسخگویی، یک امتیاز منفی بزرگ محسوب می‌شود.

بخش سوم)

```
type ComputeResponse struct {
    Operation string `json:"operation,omitempty"`
    A          *float64 `json:"a,omitempty"`
    B          *float64 `json:"b,omitempty"`
    Result     *float64 `json:"result,omitempty"`
    Error      string   `json:"error,omitempty"`
}
```

این ساختار برای محاسبه و ارسال خروجی به فرمت JSON است. و شامل نوع عملیات، عملوندها، حاصل و ارورهای محتمل است.

```
func main() {
    http.HandleFunc("/health", healthHandler)
    http.HandleFunc("/compute", computeHandler)

    port := "8080"
    log.Printf("server listening on :%s", port)

    if err := http.ListenAndServe(":"+port, nil); err != nil {
        log.Fatalf("failed to start server: %v", err)
    }
}
```

در تابع **main** از دو نقطه اتصال

Contents

No table of contents entries found.

health و **compute** پشتیبانی می‌کنیم و به پورت 8080 وصل می‌شویم.

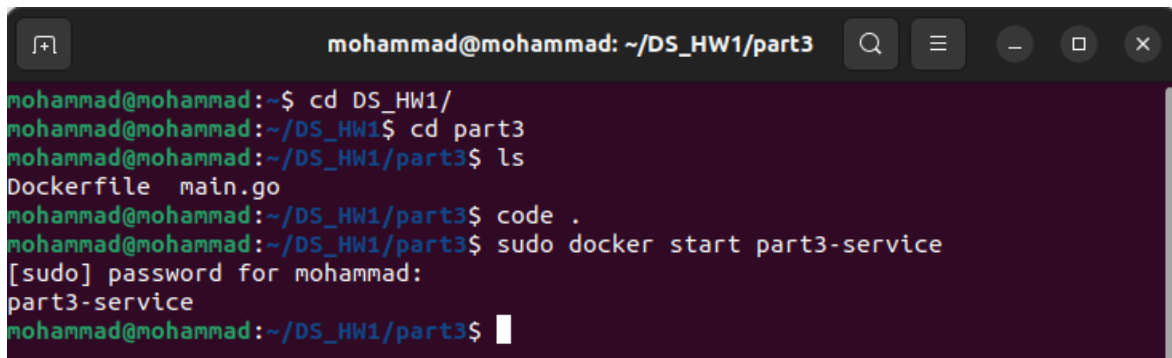
computeHandler دقیقاً شبیه worker بخش اول است ولی پاسخ به فرمت JSON است.

```
func writeJSON(w http.ResponseWriter, status int, payload any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    _ = json.NewEncoder(w).Encode(payload)
}

func healthHandler(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodGet {
        writeJSON(w, http.StatusMethodNotAllowed, map[string]string{
            "error": "method_not_allowed",
        })
        return
    }

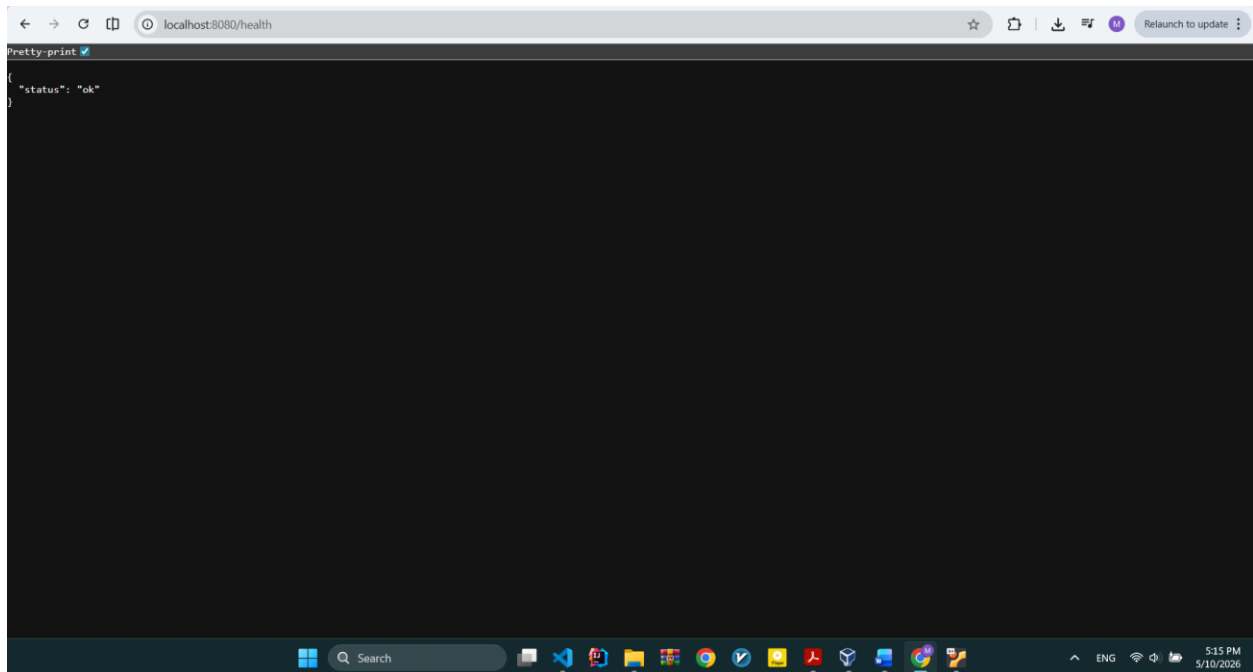
    writeJSON(w, http.StatusOK, map[string]string{
        "status": "ok",
    })
}
```

در اینجا دو متد دیگر را مشاهده می کنیم که بر روی http کار می کنند و جواب های هر درخواست را به آن ارسال می کنند.

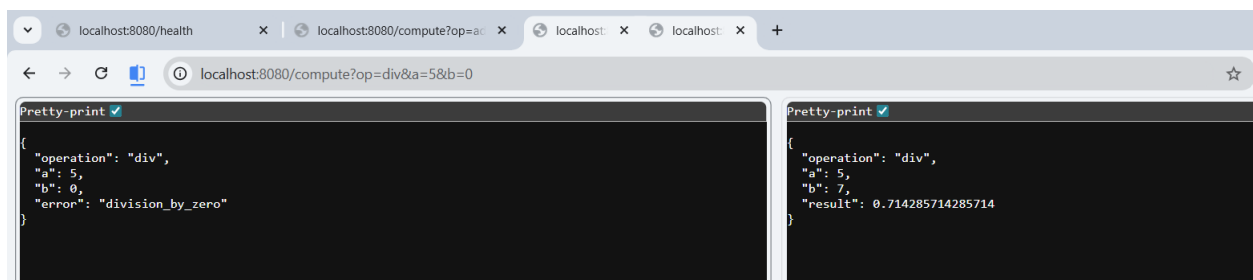


```
mohammad@mohammad: ~/DS_HW1/part3
mohammad@mohammad:~$ cd DS_HW1/
mohammad@mohammad:~/DS_HW1$ cd part3
mohammad@mohammad:~/DS_HW1/part3$ ls
Dockerfile  main.go
mohammad@mohammad:~/DS_HW1/part3$ code .
mohammad@mohammad:~/DS_HW1/part3$ sudo docker start part3-service
[sudo] password for mohammad:
part3-service
mohammad@mohammad:~/DS_HW1/part3$
```

طبق تصویر داکر را از ماشین مجازی فعال می کنیم.



در تصویر بالا اتصال ماشین میزبان به نقطه اتصال health میهمان را مشاهده می کنیم.



همانگونه که در تصویر بالا می بینیم از میزبان بطور همزمان 4 اتصال به میهمان باز کرده ایم و همه هم کار می کنند.